Chapter 15 — Storage, Data Pipelines, and I/O Performance

> *"A 10,000-GPU cluster consuming 100 GB/s of training data needs a storage system that delivers 100 GB/s reliably, every second, for weeks. A single slow checkpoint can stall $2M worth of GPU time. Storage is not secondary infrastructure — it is the rate-limiter that determines whether your GPUs compute or wait."* — AI Infrastructure Operations Principle

---

Chapter Overview

Storage is consistently the most underestimated component in AI infrastructure design. Engineers specify GPUs and networking carefully, then discover that a misconfigured NFS mount causes GPUs to idle 15% of the time waiting for data — wasting millions of dollars of compute. A 70B model checkpoint is 1.26 TB. Written synchronously, it stalls the training loop for 25 seconds per checkpoint. Written asynchronously, it adds 0.5% overhead. That 25-second difference represents tens of thousands of dollars of wasted GPU time per week at scale.

Sections in this chapter:

- 15.1 The AI Storage Hierarchy — Four Tiers
- 15.2 Distributed Filesystems — Lustre, Weka, GPFS, Tectonic
- 15.3 The Training Data Pipeline — Pre-Processing, Packing, and Prefetch
- 15.4 Checkpoint Engineering — Async, Distributed, and Validated
- 15.5 NVMe Local Storage — Staging, Caching, and KV Overflow
- 15.6 Storage Monitoring, Diagnostics, and Benchmarking
- 15.7 Key Takeaways and Review Questions

15.1 The AI Storage Hierarchy — Four Tiers

```
AI CLUSTER STORAGE HIERARCHY:

  Tier  Name              Capacity     Bandwidth      Latency    Primary Use
  ──────────────────────────────────────────────────────────────────────────
  0     GPU HBM           80-288 GB    3.35-12 TB/s   <1 μs      Active compute
  1     CPU DRAM          1-2 TB/node  400 GB/s CPU   <1 μs      Pinned batch
                                       128 GB/s PCIe              staging, KV
```

```
overflow
  2     Local NVMe        10-30 TB     50-120 GB/s    ~100 µs    Dataset shard
cache,
        (RAID-0, 8 drives)                                         checkpoint
staging
  3     Distributed FS    PBs          1-100+ TB/s    1-10 ms    Training
datasets,
        (Lustre/GPFS/Weka)                                         shared
checkpoints
  4     Object Storage    EBs          10-500 GB/s    10-200 ms  Raw data archive,
        (S3-compatible)                                            permanent
checkpoints
  ──────────────────────────────────────────────────────────────────────────────


DATA FLOW FOR A TRAINING STEP:
  Object Store → download to local NVMe (once, per dataset shard)
  Local NVMe → prefetch to CPU DRAM (ahead of time, async workers)
  CPU DRAM → DMA to GPU HBM (just-in-time, pinned memory transfer)
  GPU HBM → compute → gradients → AllReduce → optimizer
  Optimizer state → async copy to CPU DRAM → background write to Tier 3 FS

GPU STARVATION THRESHOLD:
  Data loading time must be < 3% of step time (non-negotiable target).
  If data loading = 5% of step time at 1,024 GPUs:
    Idle GPU-equivalents = 1,024 × 0.05 = 51.2 GPU-equivalents
    At $1.66/GPU-hr: 51.2 × 24 × $1.66 = $2,040/day wasted on waiting.
  Measure, set alerts, and fix before this compounds over a 30-day run.
```

15.2 Distributed Filesystems for AI Training

15.2.1 Filesystem Technology Comparison

```
AI TRAINING FILESYSTEM OPTIONS:

LUSTRE (HPC standard, most deployed):
  Architecture:    MDT (metadata targets) + OST (object storage targets)
                   Clients stripe data across multiple OSTs for parallel I/O
  Performance:     Multi-TB/s aggregate (>1,000 OSTs possible)
  Latency:         1-5 ms per metadata operation
  Scale:           Millions of parallel readers/writers
  Common at:       National labs, NVIDIA SuperPODs, large research clusters
```

```
  Limitation:      Complex to operate; requires dedicated Lustre admins

GPFS / IBM SPECTRUM SCALE:
  Architecture:    Clustered filesystem; strong POSIX consistency
  Performance:     High throughput, enterprise-grade reliability
  Use case:        Enterprise AI, financial ML, regulated industries
  Limitation:      Expensive licensing; less common in hyperscale AI

WEKA (modern parallel FS, growing fast):
  Architecture:    Distributed NVMe-native filesystem on compute nodes
  Performance:     10+ GB/s per node, linear scaling
  Latency:         <0.5 ms (NVMe-based, much lower than Lustre HDD)
  Key advantage:   Runs on standard x86 hardware with NVMe; no dedicated storage
nodes
  Common at:       Microsoft Azure AI, AWS HPC clusters, AI startups

TECTONIC (Meta internal):
  Custom-built for Meta's AI workloads (Llama-3, CLIP training)
  Optimized for:   Large sequential writes (checkpoint patterns)
  Performance:     10+ TB/s aggregate for 24K-GPU Llama clusters
  Not available:   Internal to Meta; architecture described in paper

NFS (Network File System):
  Simplest deployment; single server or small cluster
  Performance:     5-20 GB/s (single NFS server = bottleneck)
  Suitable for:    Development clusters, < 32 GPUs, inference serving
  NOT suitable:    Any serious distributed training at scale
  Failure mode:    Single NFS server becomes an IOPS bottleneck; all GPUs wait

FILESYSTEM SELECTION RULE (from r3):
  < 32 GPUs, dev/research:    NFS acceptable (simplicity wins)
  32-256 GPUs, production:    Weka or Lustre with 20+ OSTs
  256+ GPUs, hyperscale:      Custom FS, Tectonic, GPFS, or Lustre with 100+ OSTs
  Inference serving:          NFS or object storage (sequential model loads; no
burst)
```

15.2.2 Lustre Configuration — The Single Most Impactful Tuning

```
LUSTRE STRIPE CONFIGURATION — THE HIGHEST-LEVERAGE STORAGE OPTIMIZATION:

DEFAULT BEHAVIOR (stripe_count = 1):
  Every file written to Lustre lands on ONE OST.
  If you have 100 OSTs, each providing 10 GB/s:
```

```
    Theoretical aggregate: 1,000 GB/s
    Actual for any single large file: 10 GB/s  ← limited by one OST!

  A 1.26 TB checkpoint to one OST: 1,260 GB / 10 GB/s = 126 seconds!
  The other 99 OSTs are idle.

STRIPE ACROSS ALL OSTs (stripe_count = -1 = all OSTs):
  Same 1.26 TB checkpoint to 100 OSTs: 1,260 GB / 1,000 GB/s = 1.26 seconds.
  That's a 100× improvement from one configuration change.

COMMANDS:
  # Check current striping of a file or directory
  lfs getstripe /lustre/checkpoints/

  # Set stripe for a NEW directory (must do BEFORE writing data)
  lfs setstripe -c -1 /lustre/checkpoints/   # -1 = all available OSTs
  lfs setstripe -c 32 /lustre/datasets/      # 32 OSTs (appropriate for datasets)
  lfs setstripe -c 1  /lustre/metadata/      # 1 OST for small metadata files

  CRITICAL RULE: Stripe configuration applies at CREATION TIME.
  Cannot be changed retroactively for existing files.
  Set stripe on the directory BEFORE any data is written.
  Re-striping existing files requires: cp → delete original → rename.

STRIPE COUNT RECOMMENDATIONS:
  Training dataset shards (100 MB-10 GB files): stripe_count = 8-32
  Checkpoints (100 GB - 5 TB files):           stripe_count = -1 (all OSTs)
  Small scripts and configs (<1 MB):            stripe_count = 1 (default)
  Model artifacts for serving:                  stripe_count = 8-16

LUSTRE CLIENT TUNING:
  # Increase max pages per bulk RPC (better sequential throughput):
  lctl set_param llite.*.max_read_ahead_mb=256
  lctl set_param llite.*.max_read_ahead_per_file_mb=256

  # Increase dirty cache for write coalescing:
  lctl set_param llite.*.max_dirty_mb=512

  # Lock contention reduction (many-writer checkpoint):
  lctl set_param llite.*.statahead_agl=0  # Disable statahead for checkpoint dirs
```

15.3 The Training Data Pipeline

15.3.1 Why GPUs Wait for Data

```
GPU STARVATION ANATOMY:

  WITHOUT PIPELINE OPTIMIZATION:
    GPU Step 1: compute (1.4s)
    [GPU IDLE: waiting for batch to be loaded from NFS] (0.3s)
    GPU Step 2: compute (1.4s)
    [GPU IDLE: waiting...] (0.3s)

    Effective utilization: 1.4 / (1.4 + 0.3) = 82.4%
    Lost throughput: 17.6% → at 1,024 GPUs: 180 GPU-equivalents idle
    Cost: 180 × $1.66/hr × 24 = $7,172/day just from data loading stalls

  SYMPTOMS OF DATA LOADING STALLS:
    Nsight Systems: gaps before forward pass start (GPU idle, then burst of
compute)
    DCGM SM utilization: periodic dips to 0% between otherwise-dense compute
    Step time variance: some steps 2× longer than others (data loading lottery)
    Training log: "DataLoader wait" metric > 50ms per step

  GOAL: Data loading must NEVER be on the critical path.
    Target: data loading time < 3% of step time
    Mechanism: background workers prefetch N steps ahead
    The GPU should NEVER wait — batches arrive before they are needed.
```

15.3.2 Offline Pre-Processing — The Non-Negotiable First Step

```
PRE-PROCESSING PIPELINE (must be done BEFORE training starts):

STEP 1: TOKENIZE AND SERIALIZE (once, write to Lustre):
  Input: Raw text corpus (terabytes of text files)
  Process: tokenizer(text) → numpy array of token IDs
  Output: Binary shards, each ≥ 100M tokens per file
  Format: numpy memmap (.npy) or Apache Arrow (.arrow)

  # Example: pre-tokenize to memmap format
  from transformers import AutoTokenizer
  import numpy as np

  tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-3.1-70B")
```

```
  with open("/lustre/datasets/shard_000000.npy", "wb") as f:
      for doc in documents:
          token_ids = tokenizer(doc, add_special_tokens=False)["input_ids"]
          token_ids.append(tokenizer.eos_token_id)
          np.array(token_ids, dtype=np.uint16).tofile(f)

  # Result: uint16 array (32K vocab → 2 bytes/token)
  # Shard size: 100M tokens × 2 bytes = 200 MB per shard

  WHY LARGE SHARDS MATTER:
    Many small files: each open() is a Lustre metadata operation (~2ms each)
    1,024 GPUs × 4 workers each opening 1 file/step:
      4,096 metadata ops × 2ms = 8.2 seconds of metadata traffic per step!
    Large shards: open once per shard (covers millions of steps).
    Lustre metadata: nearly zero load.

STEP 2: SHUFFLE AT SHARD LEVEL:
  Fisher-Yates shuffle over the shard list.
  Each epoch: different shard ordering = different sequence of data.
  Within-shard: sequences drawn sequentially (read-sequential → fast).

  DON'T shuffle individual tokens across all shards:
    → requires random access across Lustre → kills read performance.
  DO shuffle the shard ordering + sample start within each shard.

STEP 3: VALIDATE AND DEDUPLICATE (offline, before training):
  MinHash-LSH deduplication: remove near-duplicate documents.
  Quality filtering: perplexity-based filtering (remove very high/low ppl text).
  Length filtering: remove sequences that would be heavily truncated.
  Checksum: SHA-256 each shard manifest. Load and verify at training start.
```

15.3.3 Sequence Packing — 15-25% Free Throughput

```
THE PADDING WASTE PROBLEM:

  Training data: variable-length documents
  Context window: 4,096 tokens (fixed)

  WITHOUT packing:
    Each document padded to 4,096 tokens.
    Distribution example (web-crawl corpus):
      70% of documents: < 512 tokens
```

```
      25% of documents: 512 - 2,048 tokens
       5% of documents: > 2,048 tokens

    Average document length: ~400 tokens
    Average padding per document: 4,096 - 400 = 3,696 tokens
    Padding fraction: 3,696 / 4,096 = 90.2% of tokens are padding!

    The GPU is computing 9× as much padding as real content.
    This is arithmetic intensity wasted on zeros.

  WITH SEQUENCE PACKING:
    Pack multiple documents into each 4,096-token context window.
    Average 4,096 / 400 ≈ 10 documents per context window.
    Padding: only the last document's padding (avg ~200 tokens per window).
    Packing efficiency: (4,096 - 200) / 4,096 = 95.1%

    Net throughput improvement: packing eliminates ~85-90% padding
    Effective training speedup: 15-25% at same GPU utilization.

PACKING ALGORITHM:

  # Best-fit bin packing (greedy):
  context_window = 4096
  current_bin = []
  current_len = 0
  bins = []

  for doc_tokens in shuffled_documents:
      if current_len + len(doc_tokens) + 1 > context_window:
          # Bin is full: pad to context_window, save, start new bin
          bins.append(current_bin + [PAD_ID] * (context_window - current_len))
          current_bin = []
          current_len = 0
      current_bin.extend(doc_tokens)
      current_bin.append(EOS_ID)
      current_len += len(doc_tokens) + 1

  # Each bin is exactly context_window tokens
  # Multiple complete documents separated by EOS tokens

ATTENTION MASKING FOR PACKED SEQUENCES:
  Tokens within document A must NOT attend to tokens from document B.
  Use block-diagonal attention mask:
```

```
    Doc A (tokens 0-399) → attends only to tokens 0-399
    Doc B (tokens 400-750) → attends only to tokens 400-750
    Doc C (tokens 751-4095) → attends only to tokens 751-4095

  Implementation: custom attention mask (1 = attend, 0 = block)
  FlashAttention varlen mode: flash_attn_varlen_qkvpacked_func
    Input: cu_seqlens (cumulative sequence lengths)
    No wasted compute on cross-document attention

  PyTorch implementation:
    from flash_attn import flash_attn_varlen_qkvpacked_func
    output = flash_attn_varlen_qkvpacked_func(
        qkv,
        cu_seqlens=cu_seqlens,  # [0, len_doc_A, len_doc_A+len_doc_B, ...]
        max_seqlen=context_window,
        dropout_p=0.0,
        causal=True,
    )
```

15.3.4 PyTorch DataLoader Configuration

```python
```

```python
from torch.utils.data import DataLoader, Dataset
import numpy as np
import torch

class PackedDataset(Dataset):
    """Memory-mapped dataset for fast random access without loading full shard to RA

    def __init__(self, shard_paths, context_len=4096):
        self.context_len = context_len
        # Memory-map all shards: O(1) access without loading to RAM
        self.data = np.concatenate([
            np.memmap(p, dtype=np.uint16, mode='r') for p in shard_paths
        ])
        self.n_sequences = len(self.data) // context_len

    def __len__(self):
        return self.n_sequences

    def __getitem__(self, idx):
        start = idx * self.context_len
        tokens = self.data[start : start + self.context_len + 1].astype(np.int32)
        x = torch.from_numpy(tokens[:-1]).long()  # Input
        y = torch.from_numpy(tokens[1:]).long()   # Target (shifted by 1)
        return x, y


def build_production_dataloader(dataset, batch_size, rank, world_size):
    """
    Production DataLoader configuration for LLM training.
    Critical parameters: num_workers, pin_memory, prefetch_factor.
    """
    sampler = torch.utils.data.distributed.DistributedSampler(
        dataset, num_replicas=world_size, rank=rank, shuffle=True
    )
    return DataLoader(
        dataset,
        batch_size=batch_size,
        sampler=sampler,
        num_workers=4,          # 4 worker processes per GPU (tune 4-8 for NVMe)
        pin_memory=True,        # Pin to non-pageable RAM → faster GPU transfer
        prefetch_factor=4,      # Keep 4 batches loaded ahead of GPU consumption
        persistent_workers=True, # Workers stay alive between epochs (no restart ove
        drop_last=True,         # Drop incomplete last batch (stable shapes for comp
```

```
    )

# KEY PARAMETERS:
#   num_workers=4:     Parallel prefetch. Too few: starvation. Too many: NUMA conten
#   pin_memory=True:   Skips pageable→non-pageable copy on GPU transfer. Always enab
#   prefetch_factor=4: With 4 workers: 16 batches in pipeline. Absorbs I/O spikes.
#   persistent_workers: Avoids worker process respawn overhead between steps. Always
#
# NUMA PINNING (for machines with multiple NUMA nodes):
#   numa_interleave or bind workers to same NUMA node as GPU:
#   NUMA_BIND=0 torchrun --nproc_per_node=8 train.py
```

15.3.5 DataLoader Wait Time Instrumentation

```python
```

```python
import time
import torch

class InstrumentedDataLoader:
    """Wraps DataLoader to measure per-step data loading wait time."""

    def __init__(self, dataloader):
        self.loader = dataloader
        self.total_wait = 0.0
        self.n_steps = 0

    def __iter__(self):
        it = iter(self.loader)
        while True:
            t_start = time.perf_counter()
            try:
                batch = next(it)
            except StopIteration:
                break
            self.total_wait += time.perf_counter() - t_start
            self.n_steps += 1
            yield batch

    @property
    def avg_wait_ms(self):
        return (self.total_wait / max(self.n_steps, 1)) * 1000

    @property
    def wait_fraction(self):
        """Fraction of total time spent waiting for data."""
        return self.total_wait / max(self.n_steps * step_time_estimate, 1)


# Integration in training loop:
loader = InstrumentedDataLoader(build_production_dataloader(...))
for step, (x, y) in enumerate(loader):
    loss = model(x, y)
    ...
    if step % 100 == 0:
        print(f"DataLoader wait: {loader.avg_wait_ms:.1f}ms/step")
        # Alert: if avg_wait_ms > 50ms (>3% of 1.4s step time) → fix pipeline

# ALERT THRESHOLDS:
```

```
#   < 20ms per step:    Excellent — data never limits training
#   20-50ms per step:   Acceptable — tune num_workers or prefetch_factor
#   > 50ms per step:    Problem — data loading is on the critical path
#   > 100ms per step:   Severe — check filesystem health, num_workers, stripe count
```

15.4 Checkpoint Engineering

15.4.1 The Synchronous Checkpoint Trap

```
CHECKPOINT SIZE AND SYNCHRONOUS WRITE OVERHEAD:

  CHECKPOINT COMPONENTS (70B model, AdamW, ZeRO-0 baseline):
    BF16 model weights:    70B × 2B  = 140 GB
    FP32 master weights:   70B × 4B  = 280 GB
    FP32 Adam m:           70B × 4B  = 280 GB
    FP32 Adam v:           70B × 4B  = 280 GB
    BF16 gradients:        70B × 2B  = 140 GB
    ──────────────────────────────────────────
    TOTAL:                           = 1,120 GB = 1.12 TB per checkpoint

  Checkpoint size by model (all training state):
    7B:    7B × 16B = 112 GB
    13B:   13B × 16B = 208 GB
    70B:   70B × 16B = 1,120 GB = 1.12 TB
    405B:  405B × 16B = 6.48 TB

  SYNCHRONOUS CHECKPOINT TIMING:
    Write bandwidth needed:
      To NFS at 20 GB/s:      1,120 GB / 20 GB/s = 56 seconds
      To Lustre at 100 GB/s:  1,120 GB / 100 GB/s = 11.2 seconds
      To NVMe at 80 GB/s:     1,120 GB / 80 GB/s = 14 seconds

    Checkpoint every 500 steps, step time = 2 seconds:
      Without async (Lustre): 11.2 / (500×2) = 1.12% overhead — manageable
      Without async (NFS):    56 / (500×2) = 5.6% overhead — significant
      Without async (405B):   6,480 / 100 / (500×2) = 6.5% overhead — expensive
```

15.4.2 Async Checkpointing — The Production Standard

```
ASYNC CHECKPOINT MECHANISM:
```

```
PHASE 1 — SNAPSHOT (GPU → CPU DRAM, fast):
  Copy model/optimizer state from GPU HBM to CPU DRAM.
  Non-blocking: GPU training continues on GPU compute stream.
  CPU DRAM copy finishes in ~2-5 seconds (PCIe at 128 GB/s).

PHASE 2 — BACKGROUND WRITE (CPU DRAM → Lustre, async):
  Background thread streams CPU DRAM → Lustre/NVMe.
  GPU compute continues simultaneously.
  Training is completely unaffected.

TOTAL GPU OVERHEAD: only the snapshot time (~2-5 seconds).
vs synchronous: 11-56 seconds blocked.
Net overhead: 2-5s / (500×2s) = 0.2-0.5% — negligible.

PYTORCH ASYNC CHECKPOINT IMPLEMENTATION:
```

python

```python
import torch
import threading
from pathlib import Path

def async_save_checkpoint(
    model,
    optimizer,
    scheduler,
    step: int,
    checkpoint_dir: str,
    max_local_checkpoints: int = 3,
):
    """
    Production async checkpointing: CPU snapshot + background write.
    GPU continues training immediately after the CPU copy.
    """
    import os
    ckpt_path = Path(checkpoint_dir) / f"checkpoint_step{step:08d}"
    ckpt_path.mkdir(parents=True, exist_ok=True)

    # PHASE 1: Snapshot to CPU (non-blocking GPU memory copy)
    cpu_state = {
        "step": step,
        "model": {k: v.cpu() for k, v in model.state_dict().items()},  # GPU→CPU
        "optimizer": optimizer.state_dict(),  # Already on CPU (ZeRO-3)
        "scheduler": scheduler.state_dict(),
    }
    # GPU is now free — this cpu() call was the only blocking part

    # PHASE 2: Write to storage in background thread
    def _write():
        # Write model
        torch.save(cpu_state["model"], ckpt_path / "model.pt")
        # Write optimizer
        torch.save(cpu_state["optimizer"], ckpt_path / "optimizer.pt")
        # Write metadata
        torch.save({"step": step}, ckpt_path / "metadata.pt")
        # Write manifest with checksums
        import hashlib, json
        checksums = {}
        for f in ckpt_path.glob("*.pt"):
            checksums[f.name] = hashlib.sha256(f.read_bytes()).hexdigest()
        (ckpt_path / "manifest.json").write_text(json.dumps({
```

```
            "step": step, "checksums": checksums
        }))
        print(f"Checkpoint saved: {ckpt_path}")

        # Purge old checkpoints (keep only max_local_checkpoints)
        all_checkpoints = sorted(Path(checkpoint_dir).glob("checkpoint_step*"))
        for old_ckpt in all_checkpoints[:-max_local_checkpoints]:
            import shutil
            shutil.rmtree(old_ckpt)

    thread = threading.Thread(target=_write, daemon=True)
    thread.start()
    # Return immediately — GPU training continues
    return thread  # Caller can join() before next checkpoint if needed


# FSDP DISTRIBUTED CHECKPOINT (one shard per rank — mandatory at 256+ GPUs):
from torch.distributed.checkpoint import save, load
from torch.distributed.checkpoint import FileSystemWriter, FileSystemReader

def save_fsdp_checkpoint(model, optimizer, step, checkpoint_dir):
    """
    PyTorch Distributed Checkpoint (DCP): each rank writes its own shard.
    For 256 GPUs with ZeRO-3: 256 parallel writers → 256× faster vs 1 writer.
    """
    writer = FileSystemWriter(f"{checkpoint_dir}/step_{step:08d}")
    save(
        {"model": model, "optimizer": optimizer, "step": step},
        storage_writer=writer,
    )
    # Each rank writes: 1.12 TB / 256 = 4.375 GB per rank
    # At 100 GB/s Lustre: 4.375 / 100 = 44ms per rank (parallel!)
    # Total time: ~44ms vs 11.2s single-writer = 256× faster
```

15.4.3 Three-Tier Checkpoint Retention (r3)

```
THREE-TIER CHECKPOINT STRATEGY:

TIER 1 — LOCAL NVME (fastest recovery, short retention):
  Location:     /nvme/checkpoints/ on training nodes
  Retention:    Last 2-3 checkpoints (overwrite oldest)
  Bandwidth:    80-100 GB/s write (RAID-0 × 8 NVMe)
  Write time:   1.12 TB / 80 GB/s = 14 seconds (70B model)
```

```
  Protection:   GPU failure / CUDA error / software crash
  Recovery:     <5 minutes from last checkpoint

  WHEN TIER 1 FAILS: NVMe drive failure or node loss.
  Fallback: Tier 2.

TIER 2 — DISTRIBUTED FILESYSTEM (standard, 48-hour retention):
  Location:     /lustre/project/checkpoints/ (shared across all nodes)
  Retention:    Every checkpoint from the last 48 hours
  Bandwidth:    100+ GB/s write (Lustre with proper striping)
  Write time:   1.12 TB / 100 GB/s = 11 seconds (async: ~2s GPU overhead)
  Protection:   Node failure, datacenter incident in last 48h
  Recovery:     <10 minutes from last Lustre checkpoint

  WHEN TIER 2 FAILS: Lustre corruption or datacenter event.
  Fallback: Tier 3.

TIER 3 — OBJECT STORAGE (permanent archive, full retention):
  Location:     s3://training-checkpoints/run-id/ (cloud or on-prem S3)
  Retention:    Every checkpoint from the entire run (permanent)
  Bandwidth:    10-500 GB/s (cloud S3 parallel upload)
  Upload time:  Async from Tier 2; can lag training by 15-30 minutes
  Protection:   Datacenter loss, filesystem corruption, accidental deletion
  Recovery:     30-60 minutes (download from object store)

  Upload command:
    s5cmd sync /lustre/checkpoints/step_* s3://checkpoints/run-id/ --concurrency
100

GOLDEN RULE: Always keep at least 3 valid checkpoints with different timestamps.
  If you have only 1 checkpoint and it is corrupted: restart from scratch.
  If you have 3 checkpoints: one will be valid (corruption typically affects
last).

VALIDATION BEFORE RESUME (mandatory, see Chapter 10 §10.10):
  Verify manifest checksums (SHA-256) for all .pt files before loading.
  Load-verify: check all tensor shapes match current model architecture.
  Sanity forward: run one step, compare loss to logged value at that checkpoint.
  Never resume from a checkpoint that fails any of these checks.

STORAGE REQUIREMENTS (70B model, 30-day run, checkpoint every 500 steps):
  Steps at 2s/step: 30 × 86,400 / 2 = 1,296,000 steps
  Checkpoints: 1,296,000 / 500 = 2,592 checkpoints
  Per checkpoint: 1.12 TB
```

```
  Tier 1 (last 3): 3 × 1.12 = 3.36 TB   (local NVMe)
  Tier 2 (48h):    48h × 3600s/h / (500×2s) = 172 checkpoints × 1.12 TB = 193 TB
  Tier 3 (all):    2,592 × 1.12 = 2,903 TB ≈ 2.9 PB  (object storage, archival)
```

15.5 NVMe Local Storage — Three Distinct Functions

```
NVME LOCAL STORAGE SERVES THREE DISTINCT FUNCTIONS (r3):

FUNCTION 1 — CHECKPOINT STAGING:
  RAID-0 across 4-8 NVMe drives: 50-100 GB/s per node
  Async checkpoint: GPU snapshots to CPU DRAM; background thread writes to NVMe.
  Then Tier 2 upload runs in parallel with training (NVMe → Lustre → S3).
  Benefit: NVMe write is 3-5× faster than direct Lustre write per node.
  Configuration: 8× 7.68 TB NVMe in RAID-0 = 61 TB, ~100 GB/s

FUNCTION 2 — DATASET SHARD CACHING:
  Lustre read latency: 1-5 ms per operation.
  For fine-grained random access: repeated Lustre reads = latency spikes.
  Cache hot shards on local NVMe: <0.2 ms read latency.

  Cache policy:
    LRU (Least Recently Used): evict cold shards when NVMe fills.
    Pre-warm: download next epoch's shards while current epoch trains.
    Shard affinity: pin specific shards to specific nodes (deterministic
shuffling).

  Benefit: Eliminates Lustre metadata pressure for repeated shard access.
  Cost: NVMe capacity (1-2 TB dataset cache is typically sufficient).

FUNCTION 3 — KV CACHE OVERFLOW FOR INFERENCE:
  When GPU HBM KV cache is exhausted, overflow to CPU DRAM (PCIe, 128 GB/s),
  then to NVMe if DRAM also full.

  KV overflow latency comparison:
    GPU HBM access:     <1 μs
    CPU DRAM (PCIe):    ~100 μs (DMA transfer)
    Local NVMe:         ~200 μs (NVMe latency)
    Lustre (network):   1-5 ms

  NVMe vs Lustre for KV overflow: 200 μs vs 1,000-5,000 μs = 5-25× faster.
```

```
  For inference serving: NVMe KV overflow enables longer context windows
  than HBM alone, at much lower latency than network storage.
  vLLM KV tier: --swap-space 32 (GB of CPU DRAM) → NVMe for next tier.

NVME PROVISIONING RULE:
  Size for all three functions simultaneously:
    Checkpoint staging: 3 × checkpoint_size (3 local copies = ~3 TB for 70B)
    Dataset cache:      1-5 TB depending on dataset access pattern
    KV overflow:        configurable, typically 2-8 TB
    TOTAL:              ~6-16 TB per node → 8× 2 TB NVMe drives (RAID-0)
```

15.6 Storage Monitoring, Diagnostics, and Benchmarking

15.6.1 Storage Health Instrumentation

```bash
```

```
# ─── STORAGE HEALTH DIAGNOSTIC SUITE ──────────────────────────────────────

# 1. Lustre filesystem health
lfs df -h /lustre                          # Capacity and available space per OST
lfs df -i /lustre                          # Inode usage (metadata exhaustion risk)
lctl get_param llite.*.read_bytes          # Cumulative reads from all clients
lctl get_param llite.*.write_bytes         # Cumulative writes
lfs getstripe /lustre/checkpoints/         # Verify stripe count is set correctly

# Alert: if any OST > 85% capacity → auto-purge old checkpoints immediately
# Alert: if OST imbalance > 20% → files not striped correctly

# 2. Per-step DataLoader wait time (see §15.3.5 InstrumentedDataLoader)
# Log avg_wait_ms every 100 steps. Alert at > 50ms.

# 3. Checkpoint write timing
import time
def timed_checkpoint(model, optimizer, path):
    t0 = time.time()
    torch.save({"model": model.state_dict(), "opt": optimizer.state_dict()}, path)
    elapsed = time.time() - t0
    print(f"Checkpoint write: {elapsed:.1f}s")
    if elapsed > 60:
        print("WARNING: Checkpoint exceeds 60s threshold — check Lustre health")
    return elapsed

# 4. fio — raw storage benchmark (run before job start, after cluster setup)
# Sequential read (simulate dataset loading):
fio --name=dataset_read --filename=/lustre/test --rw=read \
    --bs=128k --size=10G --numjobs=32 --group_reporting --direct=1
# Target: > 100 GB/s aggregate on Lustre with 32+ OSTs
# Alert: < 40 GB/s aggregate → stripe configuration or OST health issue

# Sequential write (simulate checkpoint):
fio --name=checkpoint_write --filename=/lustre/test --rw=write \
    --bs=1m --size=100G --numjobs=128 --group_reporting --direct=1
# Target: > 200 GB/s aggregate (for 1024-GPU checkpoint window)
# Alert: < 80 GB/s aggregate → bottleneck in write path

# NVMe local benchmark (checkpoint staging):
fio --name=nvme_write --filename=/nvme/test --rw=write \
    --bs=4m --size=50G --numjobs=8 --group_reporting --direct=1
```

```
# Target: > 80 GB/s aggregate (8× NVMe drives RAID-0)
# Alert: < 50 GB/s → drive or controller issue
```

15.6.2 Common Storage Failure Modes and Fixes

```
STORAGE FAILURE MODE DIAGNOSIS TABLE:

  Symptom                    Root Cause                  Fix
  ──────────────────────────────────────────────────────────────────────────

  GPU idle > 5% data wait   DataLoader too slow          Increase num_workers to
8;
                            (< 4 workers, no pin_memory) enable pin_memory=True;
                                                          increase
prefetch_factor=4

  Checkpoint takes > 60s    Serial write (1 rank writes  Use DCP per-rank
sharding;
  (synchronous)             entire 1.12 TB)              enable async save on CPU

  Training loss spike       Corrupted checkpoint loaded  Add SHA-256 checksum
verify
  after resume              (truncated write)            before every resume;
                                                         keep 3 recent checkpoints

  Storage read BW < 50%     Lustre stripe_count = 1     lfs setstripe -c -1 on
dirs
  of advertised spec        (default, one OST per file)  (must be set BEFORE
writing)

  Variable step time 2-3×   Lustre I/O spike from        QoS policies; dedicated
                            other concurrent jobs        storage network for
training

  High step variance        Dataset shard not in NVMe    Pre-warm NVMe shard cache
  after shard boundary      cache (first access cold)    before training begins

  Checkpoint not async;     Missing non_blocking=True    Use CPU snapshot pattern;
  GPU stalls on every ckpt  in model.state_dict() copy   threading.Thread for
write
  ──────────────────────────────────────────────────────────────────────────
```

15.7 Key Takeaways and Review Questions

```
STORAGE KEY NUMBERS (r3):

  Checkpoint sizes (full training state, 16 bytes/param):
    7B:    112 GB      13B:   208 GB      70B:   1.12 TB     405B:  6.48 TB

  Bandwidth targets (1,024-GPU cluster):
    Training data read:      > 10 GB/s aggregate       (easily met)
    Checkpoint write burst:  > 200 GB/s aggregate      (needs Lustre + stripe)
    NVMe staging (per node): 50-100 GB/s               (8× RAID-0 NVMe)
    NFS single-server:       5-20 GB/s max             (NOT for AI training at
scale)

  Latency reference:
    GPU HBM:      < 1 μs
    Local NVMe:   ~200 μs
    Lustre:       1-10 ms
    Object store: 10-200 ms

  Async checkpoint overhead:
    Synchronous (70B, Lustre):  11-25 seconds blocked → 1-5% throughput
    Async (CPU snapshot first): 2-5 seconds blocked  → 0.2-0.5% throughput

  Sequence packing gain:   15-25% effective throughput for variable-length data
  Lustre stripe speedup:   stripe_count=100 vs stripe_count=1 = 100× write BW

KEY PRINCIPLES (r3):

  1. DATA PIPELINE MUST NEVER BE ON THE CRITICAL PATH:
     Target: < 3% of step time in data loading.
     Mechanism: prefetch_factor=4 with num_workers=4-8.
     Measure: InstrumentedDataLoader avg_wait_ms per step.
     Alert at > 50ms/step. Fix at > 100ms/step.

  2. ASYNC CHECKPOINTING IS NON-NEGOTIABLE ABOVE 32B MODELS:
     Synchronous 70B checkpoint = 11-25 seconds = 1-5% wasted throughput.
     Async (CPU snapshot + background write) = 2-5 seconds = 0.2-0.5%.
     For 30-day training run: 5% overhead = 1.5 days of wasted compute.

  3. LUSTRE STRIPE CONFIGURATION IS A CREATION-TIME DECISION:
     Default stripe_count=1 → one OST → 1/100 of available bandwidth.
     Set stripe_count=-1 on checkpoint dirs BEFORE writing.
```

```
   Cannot be changed retroactively. Set it first, always.

4. THREE-TIER CHECKPOINT RETENTION PROTECTS AGAINST DIFFERENT FAILURES:
   Tier 1 (NVMe): last 2-3 checkpoints, fastest recovery (< 5 min).
   Tier 2 (Lustre): last 48 hours, protection from node failure.
   Tier 3 (Object store): entire run, permanent archive.
   Minimum: keep 3 valid checkpoints at all times. One corrupted → two backups.

5. SEQUENCE PACKING IS THE MOST UNDERUTILIZED FREE OPTIMIZATION:
   90% padding with default padding = 90% wasted compute.
   Packing reduces to < 5% padding = 15-25% effective throughput gain.
   Zero quality impact. Zero hardware change. One algorithmic change.
   Use flash_attn_varlen_qkvpacked_func for attention masking.

6. NVME LOCAL STORAGE SERVES THREE FUNCTIONS — PROVISION FOR ALL THREE:
   Checkpoint staging (RAID-0 × 8 = 50-100 GB/s fast writes).
   Dataset shard cache (avoids repeated Lustre reads, eliminates spikes).
   KV cache overflow for inference (200 μs vs 5 ms Lustre — 25× faster).
   Size: 3× checkpoint + 2-5 TB cache + 2-8 TB KV = 15-25 TB per node.
```

Review Questions

Difficulty: Foundational

Question 1 (Data Pipeline Bottleneck Diagnosis): A 512-GPU training run on LLaMA-3 70B reports 42% MFU. Nsight Systems shows clean compute-communication overlap with no obvious GPU stalls. But the team reports "some training steps take 3.2 seconds and some take 1.4 seconds" — a 2.3× variance. (a) Identify the two most likely root causes for this step-time variance pattern. (b) Write the exact code instrumentation to measure DataLoader wait time per step. (c) If DataLoader variance is confirmed, trace it to its most probable filesystem-level root cause given that the dataset is on a 48-OST Lustre filesystem with default striping.

Expected answer: (a) Two most likely causes for high step-time variance specifically: (1) Data loading stall — the DataLoader is not prefetching fast enough, so some steps must wait for the batch to arrive from Lustre. This creates bimodal step times: short steps when batch was pre-loaded (1.4s), long steps when GPU had to wait (1.4s compute + 1.8s wait = 3.2s). (2) Checkpoint write on the critical path — if a synchronous checkpoint happens every N steps, those steps are 3.2s while others are 1.4s. With 70B sync checkpoint to Lustre: 11-25 seconds (masked if batched with multiple steps). Distinguish: data stall variance is random and frequent; checkpoint variance is periodic and predictable. (b) See InstrumentedDataLoader in §15.3.5: wrap the standard DataLoader, record `time.perf_counter()` before and after `next(it)`, accumulate total wait time, divide by n_steps to get avg_wait_ms. Log every 100 steps. Alert at > 50ms. (c) Most probable filesystem-level root cause: default Lustre stripe_count=1. With 48 OSTs and stripe_count=1, every shard file reads from exactly ONE OST (10-15 GB/s). Concurrent reads

from 512 GPUs × 4 workers = 2,048 simultaneous readers hitting only 48 OSTs (averaging 43 readers per OST). When multiple workers try to read from the same OST simultaneously, they queue → I/O latency spikes → some DataLoader workers stall → batch ready too late → step time variance. Fix: lfs setstripe -c 32 (or -1) on the dataset directory, then re-copy all shards (cannot stripe existing files). With stripe_count=32, each read request is distributed across 32 OSTs → 32× higher effective per-file bandwidth → no contention.

Difficulty: Principal Level

Question 2 (Checkpoint Strategy for a 30-Day Run): You are responsible for fault tolerance on a 30-day LLaMA-3 405B training run on 4,096 GPUs (ZeRO-3, DP=512). The model checkpoint is 6.48 TB. Your Lustre storage can sustain 400 GB/s aggregate write bandwidth. (a) Calculate the minimum checkpoint interval such that checkpoint overhead stays below 1% of training throughput, assuming step time = 2.8 seconds and async checkpointing (GPU overhead = 5 seconds snapshot time). (b) At the GPU failure rate of ~1%/GPU/week, what is the expected number of hardware failures over the 30-day run? What is the maximum compute loss in GPU-hours between any two checkpoints? (c) Design the three-tier checkpoint retention policy. How many total TB of storage does your policy require?

Expected answer: (a) Async overhead per checkpoint = 5 seconds (snapshot). At 1% overhead constraint: 5s / (N × 2.8s) ⩽ 0.01 → N ⩾ 5 / 0.028 = 179 steps. Round up: checkpoint every 200 steps. Verify: 5 / (200 × 2.8) = 0.89% overhead < 1% ✓. Note: the actual Lustre write time = 6,480 GB / 400 GB/s = 16.2 seconds, but with async checkpointing this runs in the background while training continues. The GPU only sees the 5-second snapshot. (b) Hardware failures: 4,096 GPUs × 1%/week × 4.3 weeks = 176 expected GPU failures over 30 days. That's ~6 failures per day or one every ~4 hours. Maximum compute loss per failure: time between checkpoints × all GPUs. At 200 steps × 2.8s = 560 seconds = 9.3 minutes. GPU-hours lost = 4,096 GPUs × (9.3/60 hours) = 635 GPU-hours per failure. At $1.66/GPU-hr: $1,054 per hardware failure event. With 176 failures: 176 × 635 = 111,760 GPU-hours lost = ~$185,500 in wasted compute from hardware failures alone. This justifies the checkpoint infrastructure investment. (c) Three-tier retention: Tier 1 (local NVMe): last 3 checkpoints × 6.48 TB = 19.4 TB per node. With ZeRO-3 and DP=512: each of 512 ranks writes 6.48 TB/512 = 12.7 GB per rank per checkpoint. Local NVMe per node: 3 × 12.7 GB = 38 GB (trivial). Tier 2 (Lustre, 48h): 48h × 3600s/(200×2.8s) = 308 checkpoints × 6.48 TB = 1,996 TB ≈ 2 PB on Lustre. Tier 3 (object store): 30 days × 86400s/(200×2.8s) = 4,629 checkpoints × 6.48 TB = 29,995 TB ≈ 30 PB of object storage. This is a realistic number for frontier model training — 30 PB of S3 storage at $0.023/GB-month = $690,000/month storage cost. This cost justifies compression: FP16 weights only (half optimizer state) could cut to 15 PB.

Difficulty: Principal Level

Question 3 (Storage System Specification): Specify the storage architecture for a new 1,024-GPU H100 training cluster (primary use: 70–405B models). Requirements: checkpoint write latency < 60 seconds (synchronous worst case), dataset read throughput sufficient to keep GPUs fed, model weight load < 30 seconds. (a) What aggregate write bandwidth does the checkpoint requirement imply? (b) What aggregate read bandwidth does the training data requirement imply (4K seq length, step time 2 seconds, batch of 1M tokens/step)? (c) Specify the minimum viable Lustre configuration.

Expected answer: (a) Checkpoint write bandwidth: worst case = 405B model = 6.48 TB. Must write in < 60 seconds synchronous. Required BW = 6,480 GB / 60 s = 108 GB/s aggregate. Allow 20% margin: specify 130 GB/s minimum Lustre write bandwidth. (b) Training data read: 1M tokens/step × 2 bytes/token (uint16) = 2 MB per step. Step time = 2 seconds. Required read BW = 2 MB / 2s = 1 MB/s. This is trivially small — a single spinning disk could serve it! Training data IO is NOT the bandwidth constraint; it's the latency variance and metadata operations that matter. Specify: > 10 GB/s aggregate read (for prefetch overlap and metadata head room), primarily achieved by NVMe caching of hot shards, not raw Lustre bandwidth. (c) Lustre configuration: Required write BW = 130 GB/s. Modern NVMe OST provides ~10-15 GB/s write per node. Number of OSTs needed: 130 / 12 = 11 OSTs minimum. Specify: 24 OSTs (provides 288 GB/s peak, 2× headroom for burst). Per OST: 2-4 NVMe drives, 14 TB capacity each. Total raw storage: 24 OSTs × 4 drives × 14 TB = 1,344 TB ≈ 1.3 PB raw. Stripe configuration: checkpoint dirs: lfs setstripe -c -1 (all 24 OSTs). Dataset dirs: lfs setstripe -c 8 (8 OSTs per file, good balance for parallel reads). 1 MDT (metadata server) minimum; 2 MDT for HA. Networking: 200 Gbps InfiniBand from each OST to a dedicated storage network (separate from compute fabric). Switch to custom filesystem at 4,096+ GPUs where Lustre metadata operations at scale become the bottleneck (Meta moved to Tectonic at this scale).

Difficulty: Principal Level

Question 4 (Sequence Packing Implementation): Your team trains on a web-crawl dataset where 70% of documents are under 512 tokens, 25% are 512-2,048 tokens, and 5% are over 2,048 tokens. Context window is 4,096 tokens. (a) Calculate the exact padding overhead without packing (approximate each bucket as uniform, use bucket midpoints). (b) Describe the complete sequence packing algorithm at implementation level: data structure, bin-filling strategy, and termination condition. (c) Explain how cross-document attention masking must be handled, specifically: what data structure you pass to FlashAttention, and what silent failure occurs if you omit it.

Expected answer: (a) Average document length: 0.70 × 256 (midpoint of 0-512) + 0.25 × 1,280 (midpoint of 512-2048) + 0.05 × 3,072 (midpoint of 2048-4096) = 179.2 + 320 + 153.6 = 652.8 tokens. Padding per document: 4096 - 652.8 = 3,443.2 tokens. Padding fraction: 3,443.2 / 4,096 = 84.1% of tokens are padding! The GPU is computing 6.3× more padding than real tokens. (b) Bin-filling algorithm: Maintain a current_bin (list of token arrays) and current_len (integer). Iterate

over shuffled documents. For each document: if current_len + len(doc) + 1 ⩽ 4096: append doc tokens + EOS to current_bin, increment current_len. Else: pad current_bin to 4096 with PAD_ID, emit to output, reset current_bin=[doc tokens + EOS], current_len=len(doc)+1. Also track cu_seqlens: a list of cumulative sequence lengths (starting at 0, ending at 4096) indicating where each document starts and ends within the packed context. This list is the critical metadata for the attention mask. (c) FlashAttention needs cu_seqlens (cumulative sequence lengths tensor) via flash_attn_varlen_qkvpacked_func. cu_seqlens = [0, len_doc_A, len_doc_A + len_doc_B, ..., 4096]. FlashAttention uses this to apply causal attention ONLY within each document's token range. Silent failure if omitted: tokens from document B will attend to tokens from document A during the attention computation. Model will learn spurious correlations between unrelated document endings and beginnings. This degrades generalization — the model learns that every sequence start follows every sequence end, which is false. In perplexity evaluation: the model appears to predict well (it's seen training-set cross-document patterns), but on real data it performs worse. This is one of the subtlest correctness bugs in LLM training pipelines, because the loss may look similar or even slightly better (cross-document "context" provides extra signal) while real-world quality degrades.

Further Reading

1. *Efficient Large-Scale LLM Training on GPU Clusters: Data Pipeline and Storage* — Meta AI Engineering (SOSP/SIGOPS 2024) The engineering treatment of the Llama-3 training infrastructure. Storage, data pipeline, and checkpoint sections are directly applicable.
2. *Lustre Parallel Filesystem Documentation* — DDN/Whamcloud. `wiki.lustre.org` The authoritative Lustre reference. Read: stripe configuration, client tuning, and MDT load balancing sections.
3. *PyTorch Distributed Checkpoint (DCP) Documentation*. `pytorch.org/docs/stable/distributed.checkpoint.html` The per-rank sharding API. Read the `FileSystemWriter` and `FileSystemReader` sections for production use.
4. *WebDataset: Scalable PyTorch Dataset for Streaming Data*. `github.com/webdataset/webdataset` For object-store-native training data pipelines (S3 → DataLoader streaming).
5. *Data Juicer: A One-Stop Data Processing System for Large Language Models* — Alibaba DAMO Academy (2023). `arxiv.org/abs/2309.02033` The most comprehensive treatment of LLM data preprocessing pipelines: deduplication, quality filtering, tokenization, and packing at scale.

— End of Chapter 15 —

AI/ML Infrastructure from Silicon to Scale Venkat Vinjam · Principal Performance Architect © 2026 — All rights reserved